Module Code: CO658

**Examination number: 199662**

# University of Kent

# May/June 2021

# Examination Answer Booklet

Before starting this examination, you **MUST enter your Examination Number** above **(double click)**; failure to provide this means your mark cannot be recorded and **you may be awarded a 0 (zero)** for this examination.

Re-save the answer booklet to include your exam number in the file name, using the following format: [**your exam number]** [**module code] AnswerBooklet** (e.g. *123456 XX123 AnswerBooklet*).

**Answer one question per page.** You may add additional pages to this document if necessary.

| | |
|---|---|
| Question Number | 1 |

Add your answer below:

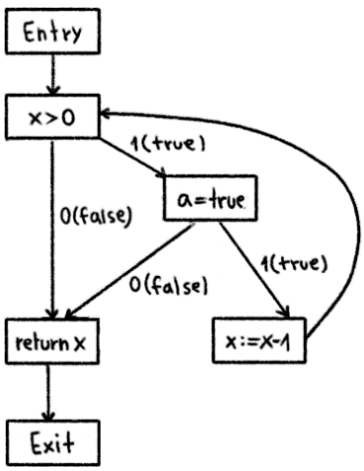
a)

Its correct to do this because it does not matter that the function call involves branching, as its not relevant for the control-flow graph. It's a good idea because each function within a block would have a graph defined separately, with each of these basic blocks will only contain information that is only relevant to the said block.

b)

A control flow graph being acyclic tells me that the function is non-recursive. This means that the control-flow graph contains no cycle loops, meaning that nodes will have to be incremented through chronologically.

c)



d)

i) The gen and kill sets for live variable analysis of each block:

EXIT: {}

BLOCK3: Gen = {z}

BLOCK2: Gen = {n}

BLOCK1: Kill = {z, n} Gen = {y, t1, t2}

ENTRY: Kill = {z, n, y}

ii) The variables that are live on exit from each block are:

EXIT: {}

BLOCK3: {z}

BLOCK2: {z, n, y}

BLOCK1: {z, n, y}

ENTRY: {z, n, y}

The algorithm used to compute this from the answer to (i) is the worklist algorithm. This algorithm essentially gathers information about the set of values using the successor to calculate the live out of the predecessor.

| | |
|---|---|
| Question Number | 2 |

Add your answer below:

a)

The concept of a register allocation by graph colouring, is to essentially make it (visually) easier to allocate registers with the use of coloured nodes. The way it is done is by having nodes have a unique colour for each node that is adjacent from one another, meaning that no nodes that are connected/adjacent have the same colour clash.

In order to colour a graph with *k* number of colours, it will first need to determine if or not the graph is fully connected, in order to find the minimum number of colours to draw out the graph (known as the chromatic number). For instance, a fully connected graph would have *k* number of colours similar to *k* number of nodes.

If a graph is not fully connected, then its possible to have nodes with fewer *k* number of colours. Firstly, the algorithm would prioritise the nodes with most neighbours, and if there are less *k* neighbours than *k* colours, then the neighbours would be coloured in as we have enough *k* colours to use.

However, if we got more *k* neighbours than *k* colours, then the algorithm will have to either allocate variables to a stack slot rather than a register or pretend that a variable is actually two variables and generate a spill when it reaches the end of the first range, to be repurposed to a second range.

b)

The assembly code, that is in text format, would need to go through 4 stages of subsequent processing, which are: assemble, link, load and run.

For assembling, the text format would be in a .s file, will have to be compiled (i.e., with NASM for an x86 machine) to convert the .s file into object data (.o file). This data would then be used for the linking process.

For linking, it would be output through a linker program (i.e., GCC). What this does is essentially concatenate two or more object files, and then assigning memory addresses to the data by editing references within the data, so it can be used within memory addresses. This is so the data can be output as an executable program now be used for the loading process.

For loading, it would have to load the program into the memory. It does this using the memory address that was compiled in the linking process with an image of the program's memory at start-up (which is essentially a capture of the program's initial execution). Usually, the process of dynamic linking will occur either within the loading process, or after execution, because the state of the memory of the machine will not be the same. So, it is there to allow the program to start in another memory address.

Finally, for running, it would have to run on the CPU. It does this through the operating system loading this from the memory and setting it to the "running" state. Again, dynamic linking may happen at this stage of execution.

| Question Number | 3 |
|---|---|

Add your answer below:

a)

i) It stores all arguments, calls the target function and retrieves the return value.

ii) It backs up a stack frame and then creates a new stack frame.

b)

i) An advantage of only using the stack pointer, is that the small amount of overhead added by the frame pointer is no longer impacting performance.

ii) A language feature which may prevent this is a debugger. Due to the fact that it makes it difficult to navigate the function, because of the lack of information on the current stack.

c)

i) Function inlining optimisation is when, instead of creating a separate set of instructions from the function into the memory, the compiler copies the code directly to the function it is being called from. To implement it, you can define the function as an inline before making calls to the function.

ii) A trade off for this optimisation is the increase in code size, causing the body of the functions to duplicate in size. It can also cause complicated performance issues if there is a larger function, as the instruction cache performance gets impacted.

iii) This statement essentially says that the compiler can partially evaluate a function before it compiles. For example, if you had a function x + 2, and you can pass x any value (If this value is 2, it makes 4). If it is inlined, the compiler will evaluate this function (so evaluate that x+2 but it will not evaluate 4) before it compiles.